

Linear Algebra :

Introduction Vectors and Matrices

Python

Vectors and their Properties

→ Linear Algebra is the branch of Mathematics that deals with Vectors & Matrices.

→ Scalar: A quantity with magnitude only.

Example: distance, speed etc

→ Vector: A quantity with both magnitude and direction.

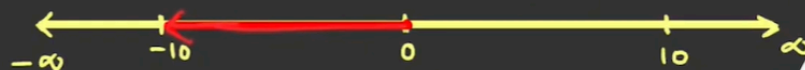
Example: displacement, velocity, acceleration, force etc

Example: A man travels 10km in +ve direction and then 20 km in -ve direction. Calculate distance & displacement.

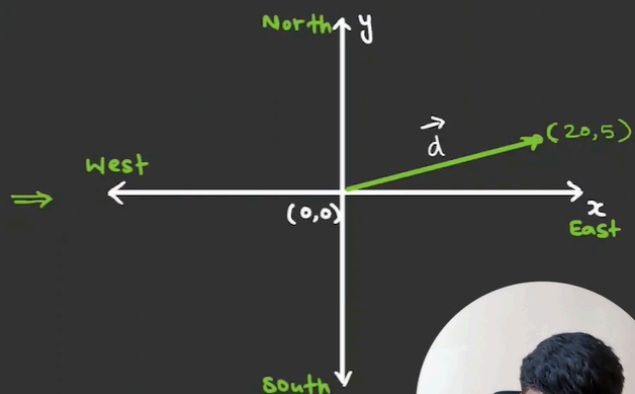
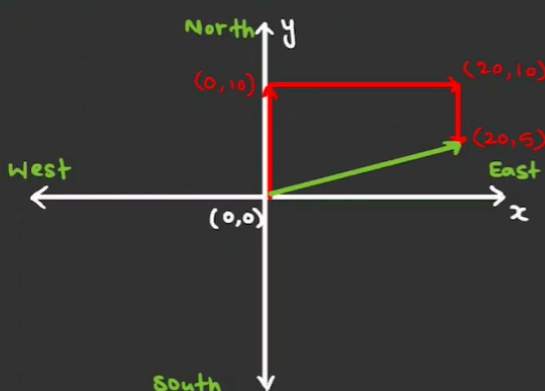


$$\text{Total distance} = 10 + 20 = 30 \text{ km}$$

$$\text{Total displacement} = +10 + (-20) = -10 \text{ km}$$



Example: A man travels 10km in North direction and then 20 km in East direction and then 5km in the South direction. Calculate distance & displacement.



→ Displacement ($\vec{d}$) = (20, 5) = $20\hat{i} + 5\hat{j} = \begin{bmatrix} 20 \\ 5 \end{bmatrix}$

$\vec{d} = |\vec{d}| \cdot \hat{d}$

Coordinate geometry

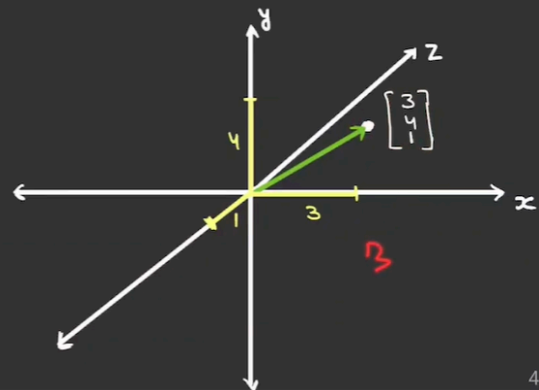
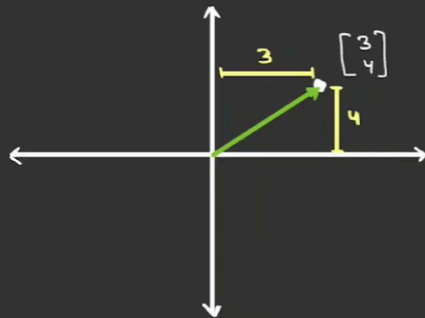
Column vector in LA

Any vector has both magnitude & direction

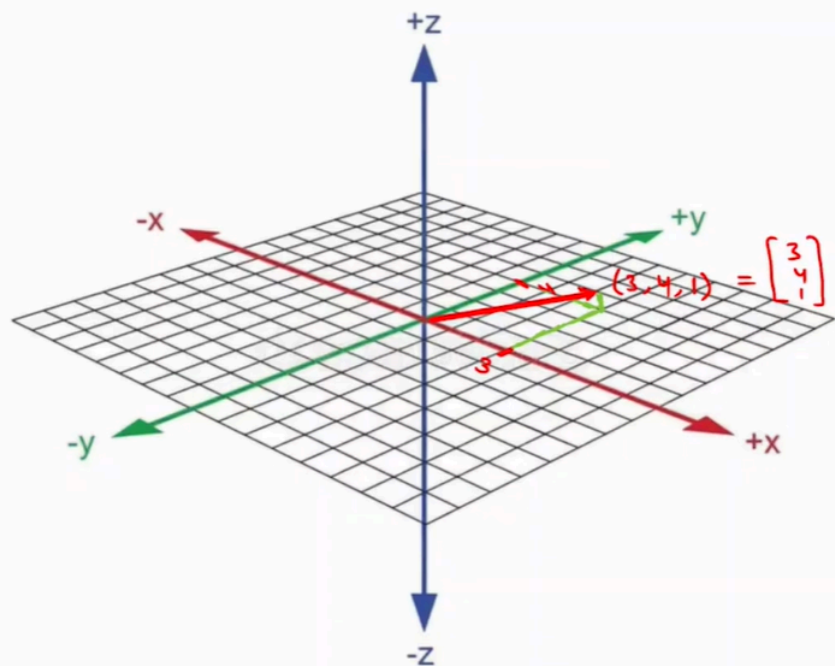
$$|\vec{d}| = \sqrt{20^2 + 5^2} = \sqrt{425}$$

$$\hat{d} = \frac{1}{\sqrt{425}} \begin{bmatrix} 20 \\ 5 \end{bmatrix}$$

* LA Intuition of a vector



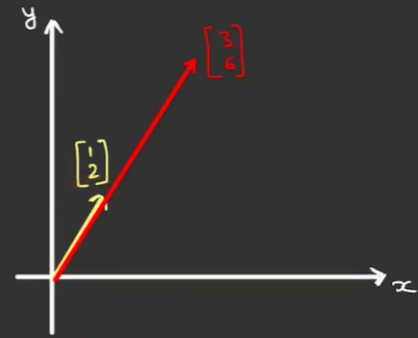
4/17



* Multiply a vector by a Scalar

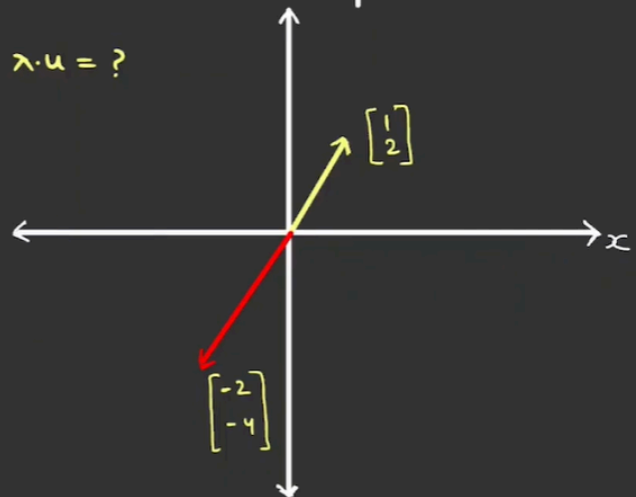
① If $u = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ and $\lambda = 3$, then $\lambda \cdot u = ?$

$$\rightarrow \lambda \cdot u = 3 \cdot \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 3 \\ 6 \end{bmatrix}$$



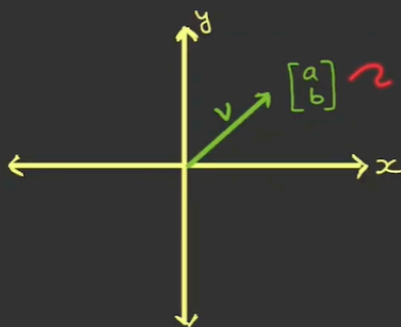
① If $u = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ and $\lambda = -2$, then $\lambda \cdot u = ?$

$$\rightarrow \lambda \cdot u = -2 \cdot \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} -2 \\ -4 \end{bmatrix}$$



* L1 and L2 Norm

$\rightarrow$ L1 and L2 norms are used a lot in ML, from loss function to Regularization.



$\rightarrow$ For $v = \begin{bmatrix} a \\ b \end{bmatrix}$

$$\text{L1-Norm} = \|v\|_1 = |a| + |b|$$

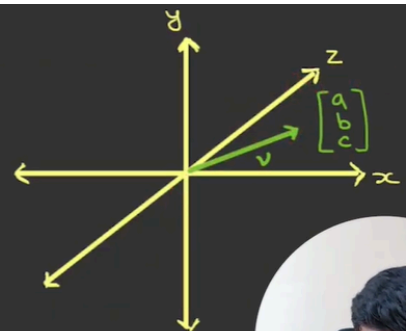
$$\text{L2-Norm} = \|v\|_2 = \sqrt{a^2 + b^2}$$

distance measured along axes

$\rightarrow$ For $v = \begin{bmatrix} a \\ b \\ c \end{bmatrix}$

$$\text{L1-Norm} = \|v\|_1 = |a| + |b| + |c|$$

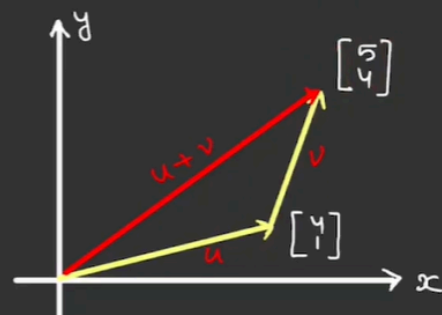
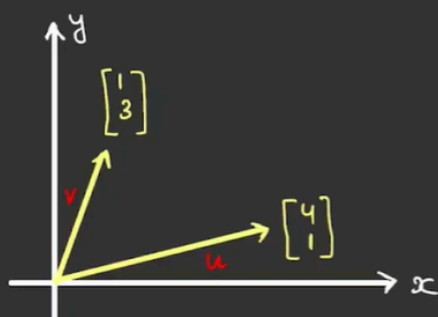
$$\text{L2-Norm} = \|v\|_2 = \sqrt{a^2 + b^2 + c^2}$$



* Sum & difference of vectors

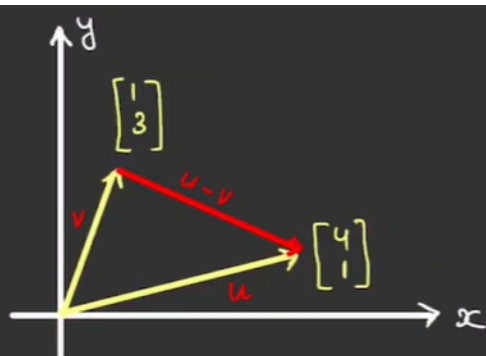
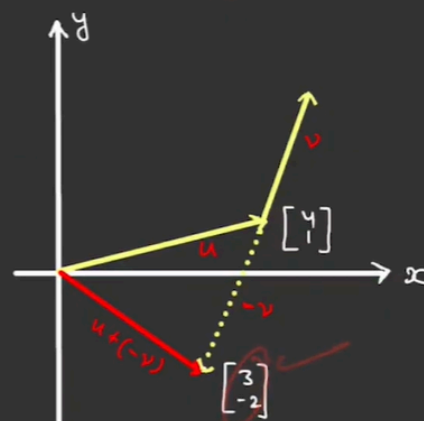
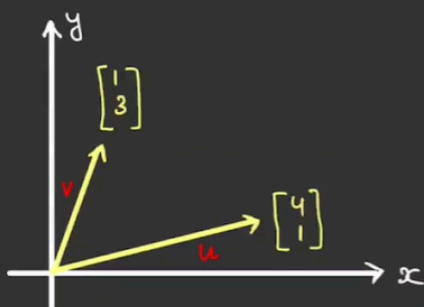
→ Calculate Sum

$$u + v = \begin{bmatrix} 4 \\ 1 \end{bmatrix} + \begin{bmatrix} 1 \\ 3 \end{bmatrix} = \begin{bmatrix} 5 \\ 4 \end{bmatrix}$$



✓ Any vector can be moved parallel to itself anywhere in space without changing its magnitude or direction

$$u - v = u + (-v) = \begin{bmatrix} 4 \\ 1 \end{bmatrix} - \begin{bmatrix} 1 \\ 3 \end{bmatrix} = \begin{bmatrix} 3 \\ -2 \end{bmatrix}$$



difference of 2 vectors is the vector drawn from tip of second vector to tip of first vector

* Dot-product of Vectors

Q. quantity = $\begin{bmatrix} 3 \\ 5 \\ 2 \end{bmatrix}$ price = $\begin{bmatrix} 10 \\ 20 \\ 30 \end{bmatrix}$ Calculate total cost of Purchase.

$$\rightarrow \text{quantity} \cdot \text{price} = \text{quantity}^T \times \text{price} = \begin{bmatrix} 3 & 5 & 2 \end{bmatrix} \times \begin{bmatrix} 10 \\ 20 \\ 30 \end{bmatrix} = 3 \times 10 + 5 \times 20 + 2 \times 30 = 160$$

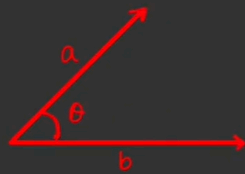
* Coding from Scratch

```
quantity = [3, 5, 2]
price = [10, 20, 30]
cost = 0
for i in range(len(quantity)):
    cost += (quantity[i] * price[i])
print(cost)
```

* coding using numpy

```
import numpy as np
quantity = [3, 5, 2]
price = [10, 20, 30]
cost = np.dot(quantity, price)
print(cost)
```

* Cosine-similarity of vectors



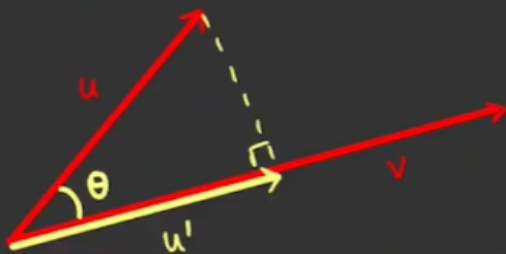
$$a \cdot b = |a| |b| \cos \theta$$



$$u \cdot u = |u| |u|$$



$$u \cdot v = |u| |v|$$



$$u \cdot v = |u| |v| \cos \theta = |u'| |v|$$

* Cosine Similarity Application

→ Cosine similarity measures how similar two vectors are by comparing their direction, not their magnitudes.

$$a \cdot b = |a||b|\cos\theta \Rightarrow \text{Cosine Similarity} = \cos\theta = \frac{a \cdot b}{|a||b|}$$

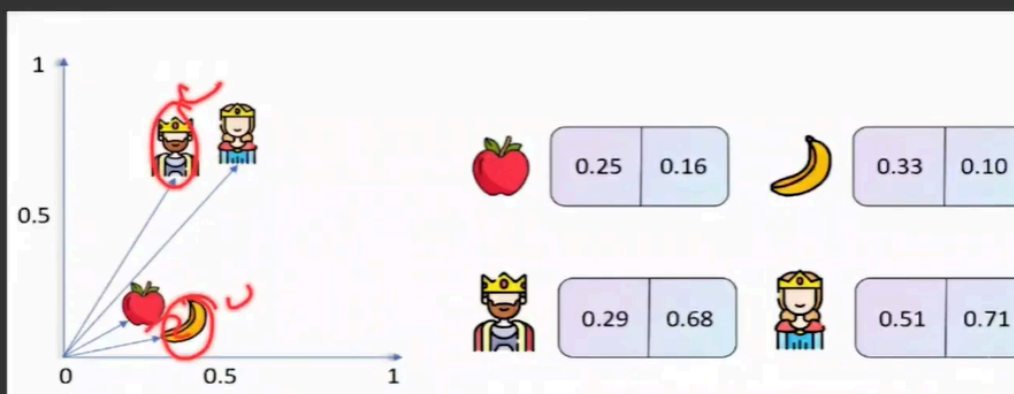
→ Cosine similarity range = $[-1, 1]$

1 → Same direction (very similar)

0 → Orthogonal (no similarity)

-1 → Opposite direction

Example: Comparing how similar 2 word embeddings are



$$\text{Cosine Similarity}(\text{King}, \text{Queen}) = \frac{\begin{bmatrix} 0.29 \\ 0.68 \end{bmatrix} \cdot \begin{bmatrix} 0.51 \\ 0.71 \end{bmatrix}}{\sqrt{0.29^2 + 0.68^2} \sqrt{0.51^2 + 0.71^2}} = 0.9759$$

$$\text{Cosine Similarity}(\text{Apple}, \text{banana}) = 0.9624$$

$$\text{Cosine Similarity}(\text{King}, \text{apple}) = 0.8263$$

→ If 2 word embeddings has cosine similarity ~ 1 , 2 words have high similarity.

→ If 2 word embeddings has cosine similarity ~ 0 , 2 words have low similarity.

→ Cosine Similarity Implementation ↘

```
import numpy as np
king = np.array([0.29, 0.68])
queen = np.array([0.51, 0.71])

similarity_score = np.dot(king, queen) /
                  (np.linalg.norm(king) * np.linalg.norm(queen))
print(similarity_score)
```

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics.pairwise import cosine_similarity

sentences = [
    "The cat sat on the mat.",
    "A cat sat on a mat.",
    "Space rockets explore distant stars."
]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(sentences)

print(f"Vocabulary : {vectorizer.get_feature_names_out()}")

similarity_matrix = cosine_similarity(X)
print(similarity_matrix.round(2))

print(f"\nSimilarity between Sentence A and B: {similarity_matrix[0][1]:.2f}")
print(f"Similarity between Sentence A and C: {similarity_matrix[0][2]:.2f}")

# output :
Vocabulary : ['cat' 'distant' 'explore' 'mat' 'on' 'rockets' 'sat' 'space' 'stars'
'the']
[[1.  0.71 0.  ]
 [0.71 1.  0.  ]
 [0.  0.  1.  ]]

Similarity between Sentence A and B: 0.71
Similarity between Sentence A and C: 0.00
```

* Relationship between Matrix & Vectors

$$A = \begin{bmatrix} 5 & 2 & 3 & 4 \\ 2 & 5 & 3 & 6 \\ 1 & 2 & 3 & 2 \end{bmatrix}$$

→ Give a matrix A of shape $(3,4)$

→ we can say that

① A is a stack of 4 column vectors

$$\begin{bmatrix} | & | & | & | \\ c_1 & c_2 & c_3 & c_4 \\ | & | & | & | \end{bmatrix}$$

② A is a stack of 3 row vectors

$$\begin{bmatrix} - & r_1 & - \\ - & r_2 & - \\ - & r_3 & - \end{bmatrix}$$